

OPEN SOURCE · MCP SERVER

Salesforce Cloud MCP Server

Bringing live Salesforce data into a Claude conversation —
for Sales, PreSales, Marketing, and Customer Success, in one server.

Sales Cloud

Service Cloud

Marketing Cloud

Model Context Protocol

Python

Mahesh Sane

Built on Claude · Anthropic's Model Context Protocol

THE PROBLEM

Four teams, four dashboards, one customer

A Sales rep checks pipeline in one report. PreSales pulls deal detail from another tab. Marketing builds a campaign ROI dashboard. Customer Success cross-references renewal and case reports by hand every Monday morning. All four are describing the same underlying Salesforce org — just through different, disconnected views of it.

Every one of those views takes a login, a click path, and a mental switch away from the actual conversation someone's trying to have — with a customer, a manager, or a teammate.

4

functions this serves — Sales, PreSales, Marketing, CS

23

ready-made tools across those functions

1

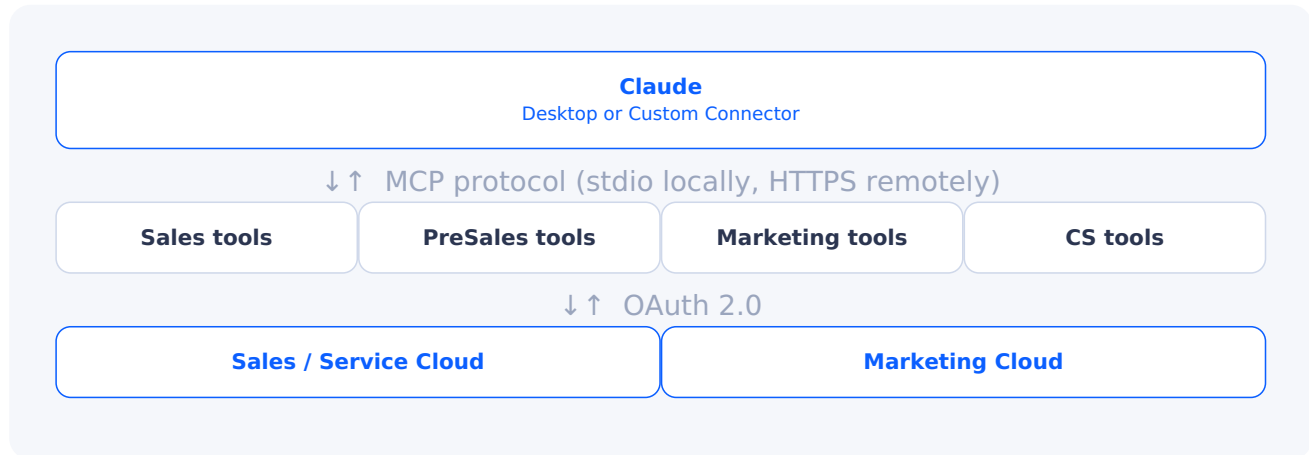
conversation to get any of them an answer

This project asks a simple question: what if the CRM answered you directly, in plain language, inside the conversation you're already having — instead of the other way around?

THE SOLUTION

One MCP server, two ways to run it

Built on Anthropic's **Model Context Protocol** — an open standard for connecting an AI assistant to real, live data. Claude doesn't get a snapshot or a training-time guess about your Salesforce org; it calls the same REST API your browser does, with your own OAuth session, in real time.



Local variant — runs on one person's laptop via Claude Desktop, session lives in a local file. Nothing to host.

Remote variant — one container, deployed to AWS, Google Cloud, or Azure, shared by a team as an always-on Custom Connector.

Built for the whole funnel, not one team's slice

Sales

Pipeline by stage, top open deals, rolling forecast, lead triage — and logging a call note straight back to Salesforce.

"Summarize my pipeline by stage, then show me the 10 biggest open deals."

PreSales

Full deal context before a call — contacts, quoted products — or the entire account in one shot before a QBR.

"Give me the account 360 for Acme Corp before my demo tomorrow."

Marketing

Campaign ROI, cost-per-lead, member attendance, and which channels are actually converting.

"Which campaigns this quarter had the best ROI?"

Customer Success

Open cases by priority, upcoming renewals, and a best-effort at-risk view combining both.

"Which accounts are at risk in the next 60 days?"

Every tool is documented for what it's built on — standard Salesforce fields where possible, with a clear note on where an org's custom fields (health score, NPS, renewal risk) need one small addition. The project ships with a contributor's guide for exactly that.

Engineered to be extended, not just used

- **OAuth 2.0 Authorization Code + PKCE** for Sales/Service Cloud — a real browser login, a refresh token, no hardcoded credentials anywhere in source.
- **A separate, correctly-scoped connection for Marketing Cloud** — it's a genuinely different Salesforce product with its own API, and the code treats it that way instead of pretending it's the same system.
- **One tool file per business function** (sales, presales, marketing, customer success) sharing a single generic SOQL/REST client — adding a new tool never touches auth or another team's file.
- **Read tools and one clearly-marked mutating tool**, with the convention documented so anyone contributing keeps that split visible.
- **Two runnable entrypoints, one codebase** — a stdio process for local use, a Streamable-HTTP service with a health check and access control for cloud deployment, sharing every tool and client underneath.
- **MIT licensed**, with a contributor's guide, an environment template, and a fictional demo dataset so anyone can try the tool shapes before connecting a real org.

From a sandbox login to a live conversation

- 1 Spin up a Salesforce sandbox**
Free Developer Edition — never start against production.
- 2 Create a Connected App and authenticate once**
A browser login, a refresh token saved locally — no server-side signup.
- 3 Run it locally and try real prompts**
Confirm your org's picklist values and custom fields match before scaling up.
- 4 Move to production, containerize, deploy**
AWS App Runner, Google Cloud Run, or Azure Container Apps — same steps, your choice of cloud.
- 5 Connect it in Claude and share it out**
One Custom Connector, tested with one person first, then rolled out to the team.

Open source. MIT licensed. Built to be forked.

Full source, setup guide, and contributor docs on GitHub.

github.com/maheshsane/salesforce-mcp-server

Built by Mahesh Sane — connect on LinkedIn to talk Customer Success, PreSales tooling, or what's next for this project.